# Lab Assignment 6 (150 points)

## Matrix Multiplication Using Systolic Arrays

### Guideline

This lab may be completed individually or in pairs. Simulate and synthesize the complete design in Vivado. Each student must understand the floating-point datapath, the systolic dataflow, and the timing of the operation. Report the synthesized resource utilization. No bitstream or board demonstration is required.

### Objective

Design, simulate, and synthesize a 3 × 3 matrix multiplier using a systolic array of nine multiply-and-accumulate (MAC) processing elements. Each MAC performs arithmetic using Tesla CFloat8 E4M3 with a configurable exponent bias.

### Description

A systolic array is a network of processing elements that compute and pass data through the system in a regular pattern. An input value is reused as it moves through neighboring processing elements. In this lab, matrix A values move horizontally to the right and matrix B values move vertically downward through registered connections.

### a. Systolic Array Architecture

Implement C = AB for two 3 × 3 matrices. Use zero-based indices throughout this assignment. The mathematical target is:

$$C = AB, \qquad c_{ij} = \sum_{k=0}^{2} a_{ik} b_{kj}, \quad 0 \le i, j \le 2$$

There are 27 scalar multiplications. Instantiate nine MAC processing elements in a 3 × 3 grid. Each processing element PE(i,j) accumulates three products to produce one output $c_{ij}$, in the order k = 0, 1, 2.

PE(i,j) receives A from its left and B from above, updates its local 8-bit CFloat8 accumulator, and registers the unchanged 8-bit A and B encodings for its neighbors. Matrix A operands move horizontally to the right, and matrix B operands move vertically downward. Each registered link advances one PE per clock cycle. The accumulator stays in its own PE.

Nine MACs operate in parallel with staggered operand movement. Cycle 0 is a separate setup cycle. Computation takes place in cycles 1 through 7, with register updates on the rising clock edge of each cycle. Section d defines the control and timing requirements.

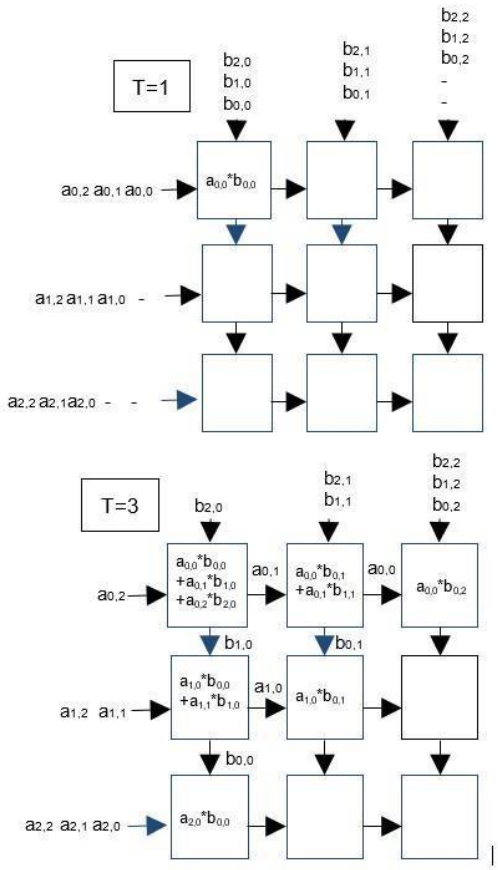



Figure 1. Operand placement during computation cycles T = 1 and T = 3.

Figure 1 illustrates data movement and partial dot products. In cycle 1, a00 and b00 enter the top-left MAC. In cycle 2, these operands move to neighboring MACs as the next inputs arrive. The diagram also shows the partial dot products in cycle 3. The process continues until the last result is completed in cycle 7. A dash at a boundary indicates an explicit 8'h00 padding value. The input schedule in Section d defines the complete boundary streams.

**Restriction:** Do not use procedural for loops to perform matrix multiplication in the design. Implement the arithmetic in synthesizable Verilog RTL.

## b. Tesla CFloat8 E4M3

Tesla CFloat8 is an 8-bit floating-point representation with a configurable exponent bias. E4M3 means the format has one sign bit, four exponent bits, and three mantissa bits. For this assignment, all matrix inputs, intermediate products, accumulator values, and outputs use this format. The unsigned 6-bit bias is an input to the system and should be latched during setup. The latched bias is shared by all nine MAC units throughout the operation.

| Field | Bits | Meaning |
|---|---|---|
| Sign s | [7] | 0 for positive and 1 for negative |
| Exponent E | [6:3] | Unsigned 4-bit field from 0 through 15 |
| Fraction F | [2:0] | Unsigned 3-bit field from 0 through 7 |

For bias b, decode zero and normal inputs using the equations below. Normal values have an implicit leading 1. Denormalized number handling (exponent = 0 and nonzero fraction) will not be tested.

$$x = 0 \qquad E = 0,\ F = 0 \ \text{(Zero)}$$

$$x = (-1)^{s}\left(1 + \frac{F}{8}\right)2^{E-b} \qquad E > 0 \ \text{(Normal)}$$

Both 8'h00 and 8'h80 are accepted as zero inputs. Every nonzero exponent pattern, including E = 15, represents a finite normal value. For the Tesla CFloat8 representation, no FP8 encoding is reserved for infinity or NaN.

### Examples

At bias = 10, 8'h50 represents +1 (normal) and 8'hD4 represents −1.5 (normal). For a fixed nonzero encoding, increasing the bias by one halves its value.

## c. CFloat8 MAC Operation

Each MAC receives two 8-bit CFloat8 E4M3 operands and uses an 8-bit CFloat8 multiplier, an 8-bit CFloat8 adder, and an 8-bit CFloat8 accumulator register. All arithmetic uses the shared operation bias.

Start the accumulator at positive zero (8'h00). For each valid operand pair, multiply the operands and round and saturate the product to CFloat8. Add that 8-bit product to the current accumulator, then round and saturate the sum before storing it in the accumulator. Process three operand pairs in the order set by the input schedule. After the third valid product, the accumulator contains the output.

In Figure 2, P is the rounded 8-bit product, ACC is the current accumulator value, and ACC_next is the rounded sum stored in the accumulator.

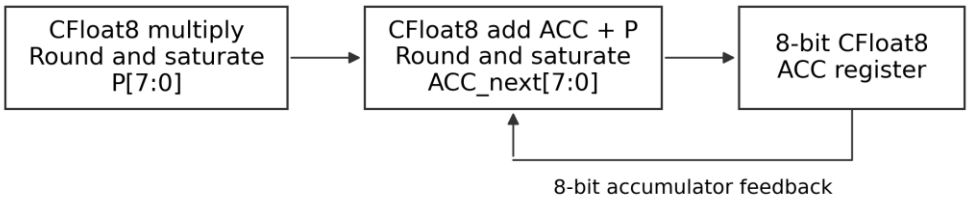



Figure 2. CFloat8 MAC datapath. A and B also pass unchanged through 8-bit forwarding registers.

Round and saturate each multiplication and addition separately before using its result in a subsequent operation. Intermediate rounding and saturation may affect the final matrix result. Do not combine the operations into a fused multiply-add.

The multiplier, adder, and rounding logic may be combinational between register updates. Each MAC updates its accumulator once per cycle when it receives a valid operand pair.

### Rounding and range behavior

Round every multiplication and addition separately to nearest, ties-to-even. Apply the magnitude rule symmetrically to positive and negative results.

Saturate overflow to 7F for positive values or FF for negative values. Encode exact zero results as positive zero (00). No encoding represents infinity or NaN.

## d. Interface and Timing

### Top-Level Interface

Use the module declaration provided in matrix.v. This lab includes an autograded component, so preserve the module name and all port names, directions, and widths. Outputs may be declared reg if driven by procedural logic. The required Matrix interface is shown below. Each matrix element is a separate 8-bit input, for 18 matrix inputs in total. The unsigned 6-bit bias is shared throughout an operation. The array generates the staggered boundary streams internally. Output cIJ represents row I and column J of C.

```
module Matrix (
    input  wire       clk,
    input  wire       reset,
    input  wire       start,
    input  wire [5:0] bias,
    input  wire [7:0] a00, a01, a02, a10, a11, a12, a20, a21, a22,
    input  wire [7:0] b00, b01, b02, b10, b11, b12, b20, b21, b22,
    output wire       busy,
    output wire       done,
    output wire [7:0] c00, c01, c02, c10, c11, c12, c20, c21, c22
);
    // Your implementation
endmodule
```

Students may choose their own internal module organization and arithmetic implementation. All arithmetic must be implemented using synthesizable Verilog RTL. Behavioral addition (+) and multiplication (*) operators are permitted. Built-in floating-point types (real, shortreal) and vendor floating-point IP are not permitted in the design.

### Operation protocol

| Event | Required behavior |
|---|---|
| Reset | Synchronous, active high. On a rising clock edge with reset = 1, abort any operation; clear all PE accumulators and operand registers, clear c00 through c22 to 00, and set busy = 0 and done = 0. Reset has priority over start. |
| Accept start | When reset = 0 and busy = 0, a rising clock edge with start = 1 begins setup cycle 0. Latch all a00–a22 and b00–b22 inputs and bias into registers for the entire operation; clear PE state and outputs, set busy = 1, and set done = 0. Computation begins in cycle 1. |
| Compute | The next seven cycles are computation cycles 1 through 7. On each rising clock edge, the latched matrices feed the scheduled MAC updates and registered operand movement, as shown in the input schedule below. Retain the latched matrices and bias throughout computation. Changes to the external matrix inputs or bias must not affect the operation. Ignore start while busy = 1, including cycle 7. There is no stall input. |
| Complete | Immediately after the rising clock edge of computation cycle 7, all nine outputs contain their final FP8 encodings, busy = 0, and done = 1. Outputs and done remain unchanged until reset or the next accepted start. |
| Restart | A new setup cycle may begin on the next rising clock edge after completion. No reset is needed between operations. Pulse start for one cycle and deassert it during computation. |

On the rising clock edge of an accepted start in setup cycle 0, latch all 18 matrix inputs and bias into registers. Retain these captured values throughout computation cycles 1–7. Use only these latched values for the operation, so changes to the external matrix inputs or bias during computation do not affect the result. Outputs remain 00 throughout computation cycles 1 through 6.

### Input Schedule

During computation cycles 1–7, the top level generates the boundary values below from the matrices latched during setup. The table entries refer to these stored values, not the live input ports. Each 00 is an explicit 8'h00 zero-padding value.

| Cycle | A row 0 | A row 1 | A row 2 | B column 0 | B column 1 | B column 2 |
|---|---|---|---|---|---|---|
| 1 | $a_{00}$ | 00 | 00 | $b_{00}$ | 00 | 00 |
| 2 | $a_{01}$ | $a_{10}$ | 00 | $b_{10}$ | $b_{01}$ | 00 |
| 3 | $a_{02}$ | $a_{11}$ | $a_{20}$ | $b_{20}$ | $b_{11}$ | $b_{02}$ |
| 4 | 00 | $a_{12}$ | $a_{21}$ | 00 | $b_{21}$ | $b_{12}$ |
| 5 | 00 | 00 | $a_{22}$ | 00 | 00 | $b_{22}$ |
| 6 | 00 | 00 | 00 | 00 | 00 | 00 |
| 7 | 00 | 00 | 00 | 00 | 00 | 00 |

Cycle 0 is setup, with no MAC update. The testbench supplies matrix inputs for setup cycle 0 only. Padding flushes the registered operand paths.

### Completion timing

All nine outputs and done must be valid immediately after the rising clock edge of computation cycle 7, once registered updates have settled. No additional cycle is allowed.

## e. Verification

We recommend creating a Verilog testbench that compares all nine 8-bit outputs exactly and checks reset, start acceptance, busy behavior, done timing, output hold, and restart. Recommended tests include changing the external matrix inputs and bias after setup to verify that the result still uses the latched values. Other recommended tests cover signed inputs, both zero encodings, normal values, rounding boundaries, and saturation.

The provided matrix_tb.v is an example testbench with three sample cases. These cases are illustrative and are not an exhaustive list of autograder tests.

While the 6-bit bias input can represent values from 0 through 63, the autograder will test only bias values 9 through 12, inclusive. You do not need to verify bias values outside this tested range. Autograder cases use zero and normal inputs. Each nonzero exact product and sum is at least as large in magnitude as the smallest normal value, so denormal results and underflow are outside the tested cases.

## Resource Utilization

Synthesize the complete Matrix design in Vivado using FPGA part XC7A35TCPG236-1. Report the post-synthesis counts of LUTs, flip-flops, latches, and DSPs for the complete top-level design. If a resource count is zero, report 0 rather than leaving the entry blank.

| LUT Usage | Flip-Flop Usage | Latch Usage | DSP Usage |
|---|---|---|---|
| | | | |

## Submission Details

Zip all source and testbench files, along with the resource utilization table into a single folder using one of the following naming schemes: EID_Lab6.zip for an individual submission, or EID1_EID2_Lab6.zip for a partner submission.

Submit all Verilog source files and the completed resource utilization table.

## Checkout Details

Be prepared to discuss any part of your implementation. No board demonstration is required.

References: [1] Tesla Dojo Technology, p. 4